Cours MERN Stack – Séance 3

Persistance des Données avec MongoDB Atlas et Mongoose (ODM)

Dr. Abdelweheb GUEDDES – École Polytechnique de Sousse – 2026–2027

Objectif	Connecter l'API à MongoDB Atlas via Mongoose, maîtriser l'asynchronisme (`async/await`), et persister des relations avec `.populate()`.	**Durée**	3 heures (en laboratoire)
Prérequis	Séance 2 validée (Architecture SoC `routes/` + `controllers/`, opérations CRUD).	**Livrable**	Compte-rendu PDF individuel sur la plateforme avant la séance 4.

Comment travailler avec ce document. Vous avancez selon le cycle d'apprentissage actif :

1. **Prédisez** : anticipez l'impact de la persistance et de l'asynchronisme **avant** d'exécuter le code.
2. **À vous** : appliquez les configurations, modèles et requêtes dans votre éditeur (VS Code) et testez avec Postman.
3. **Point de contrôle** : vérifiez les réponses JSON (codes 200, 201, 400, 404, 409) et la console MongoDB Atlas.

Grâce à l'architecture SoC mise en place en Séance 2, **vos routeurs ne subiront aucune modification** : seule la logique interne des contrôleurs migrera des tableaux volatils vers Mongoose !

1 Pourquoi une Base de Données ? Du Volatile au Persistant

Lors de la séance précédente, nos articles et utilisateurs étaient stockés dans des tableaux JavaScript (`let articles = [...]`). Bien que pratique pour apprendre, ce mécanisme souffre d'un défaut rédhibitoire pour une application de production : la **volatilité totale**.

Prédisez. Que se passe-t-il si vous effectuez 10 requêtes `POST` pour créer des articles, puis que vous redémarrez votre serveur avec `Ctrl+C` suivi de `npm run dev` ? Que recevra un client exécutant un `GET /api/articles` immédiatement après ?

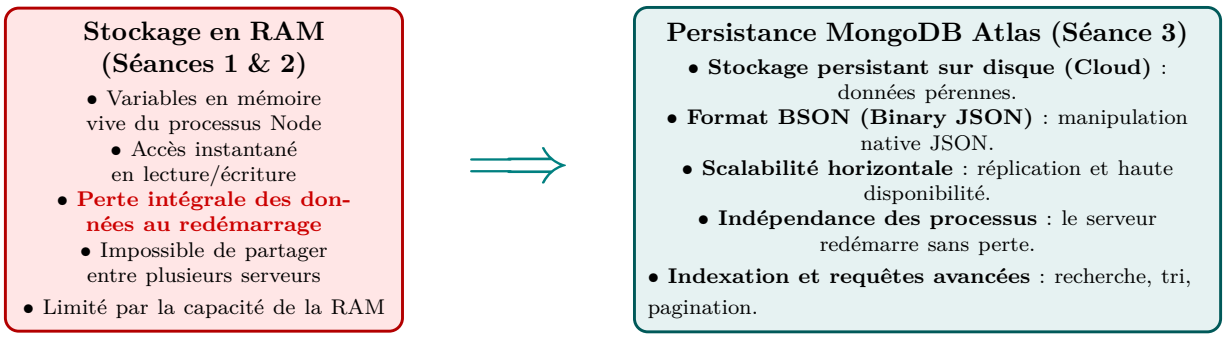


FIGURE 1 – Comparaison entre stockage éphémère en mémoire vive et persistance sur MongoDB.

2 Mise en Place de MongoDB Atlas & Sécurisation des Secrets

MongoDB Atlas est la version Cloud managée de MongoDB. Suivez rigoureusement ces 4 étapes de configuration :

1. **Création du Cluster gratuit :** Rendez-vous directement sur `https://cloud.mongodb.com` (ou

`https://www.mongodb.com/cloud/atlas/register`), créez un projet puis déployez un cluster gratuit **M0 Sandbox** (région AWS la plus proche, ex. *Frankfurt*).

2. **Sécurité utilisateur (Database Access) :** Créez un utilisateur de base de données (ex : nom `admin_mern`, mot de passe robuste `PolyMern2026!`). Attribuez-lui les droits **Read and write to any database**. **Conservez précieusement ce mot de passe !**
3. **Accès Réseau (Network Access) :** Ajoutez une adresse IP. Pour le laboratoire et le développement, choisissez **Allow Access From Anywhere** (`0.0.0.0/0`).
4. **Chaîne de connexion (Connection String) :** Cliquez sur **Database** → **Connect** → **Drivers (Node.js)**. Copiez l'URI fourni, du format suivant :

```
mongodb+srv://admin_mern:<password>@cluster0.xxxxx.mongodb.net/blogDB?retryWrites=
    true&w=majority
```

Règle absolue de sécurité en entreprise : Ne commitez JAMAIS vos mots de passe et chaînes de connexion dans votre dépôt Git ! Le fichier `.env` contenant vos secrets doit figurer impérativement dans votre fichier `.gitignore`.

3 Architecture Mongoose : Schémas, Modèles et Relations NoSQL

Mongoose est un **ODM (Object Data Modeling)**. Il s'intercale entre votre code Express et MongoDB pour apporter structure, rigueur et validation à un univers NoSQL naturellement sans schéma (*schema-less*).

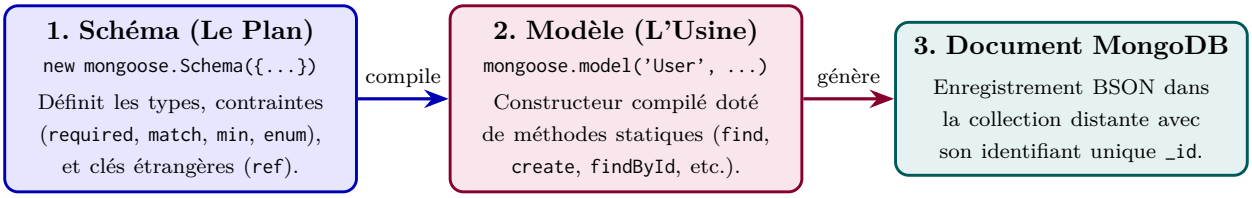


FIGURE 2 – Du Schéma Mongoose au Document dans MongoDB Atlas.

Exemple de Schéma avec Validation de Forme (Regex), Contrainte Numérique et Énumération

Voici un exemple de schéma illustrant les trois contraintes les plus courantes d'une API :

1. **Vérification de forme d'email par Regex** avec la propriété `match` ;
2. **Contrainte numérique minimale sur l'âge** avec `min: 4` ;
3. **Énumération stricte** avec `enum` restreignant le champ `role` à un ensemble fini.

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: [true, 'Le nom est obligatoire'], trim: true },
  email: {
    type: String,
    required: [true, "L'email est obligatoire"],
    unique: true,
    lowercase: true,
    trim: true,
    // Validation de forme d'email via Regex (match) :
    match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, "Format d'email invalide (ex: nom@domaine.com)"]
  },
  age: {
    type: Number,
    // Age minimal autorise fixe a 4 ans :
    min: [4, "L'age minimum autorise est de 4 ans"],
    max: [120, "L'age maximum autorise est de 120 ans"]
  },
  role: {
    type: String,
    // Enumeration : seules ces valeurs sont acceptees
    enum: {
      values: ['user', 'admin'],
      message: '{VALUE} n\'est pas un role autorise'
    },
    default: 'user'
  }
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

Modélisation des Relations en NoSQL : 1 :1, 1 :N et N :N

Contrairement aux bases relationnelles SQL fondées sur les clés étrangères et les jointures strictes, MongoDB propose deux approches complémentaires : l'**imbrication** (*embedding*) et le **référencement** (*referencing* avec `ObjectId`).

1. Relation One-to-One (1 :1) : Un utilisateur possède un unique profil détaillé (ou une adresse).

— *Approche imbriquée (recommandée pour la performance) :*

```
const userSchema = new mongoose.Schema({
  name: String,
  profile: { bio: String, website: String, avatar: String } // Sous-document
    embarque
});
```

— *Approche par référence (si le document cible est lourd ou indépendant) :*

```
profile: { type: mongoose.Schema.Types.ObjectId, ref: 'Profile', unique: true }
```

2. Relation One-to-Many (1 :N) : Un auteur (`User`) rédige plusieurs articles (`Article`). Dans chaque article, on stocke la référence de son auteur via son `_id` :

```
// Dans models/articleModel.js
const articleSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: { type: String, required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
});
```

Lors de la consultation, Mongoose reconstitue l'objet utilisateur à la volée avec `.populate()` :

```
const articles = await Article.find().populate('author', 'name email');
```

3. Relation Many-to-Many (N :N) : Un article possède plusieurs tags / catégories (`Tag`), et un tag qualifie plusieurs articles. On utilise un **tableau d'identifiants `ObjectId`** :

```
// Dans models/articleModel.js
const articleSchema = new mongoose.Schema({
  title: { type: String, required: true },
  tags: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Tag' }] // Tableau de
    references
});
// Reconstitution dynamique de la collection complete de tags :
const article = await Article.findById(id).populate('tags', 'label color');
```

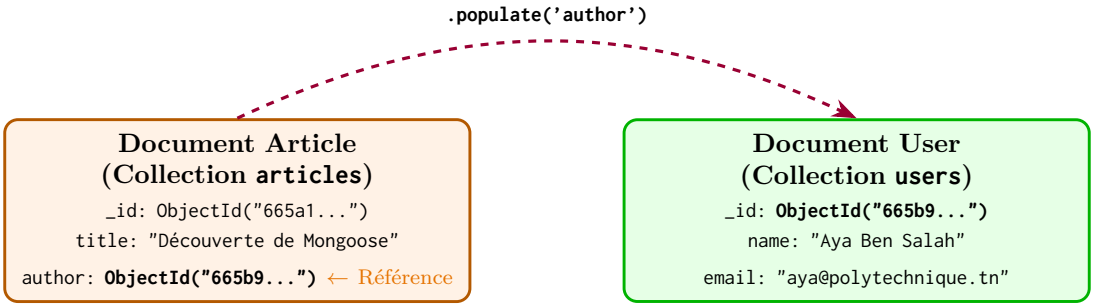


FIGURE 3 – Reconstitution dynamique de la relation One-to-Many via `.populate()`.

4 Le Paradigme Asynchrone : Des Promesses à `async/await`

Maintenant que nos schémas et relations sont définis, nous allons écrire le code pour y accéder. Contrairement à la lecture d'une variable locale en mémoire vive qui est quasi-instantanée, toute opération avec MongoDB Atlas nécessite un aller-retour réseau (I/O). Pour que le serveur Node.js ne soit jamais bloqué en attendant la réponse de la base, Mongoose retourne des **Promesses (Promises)**.

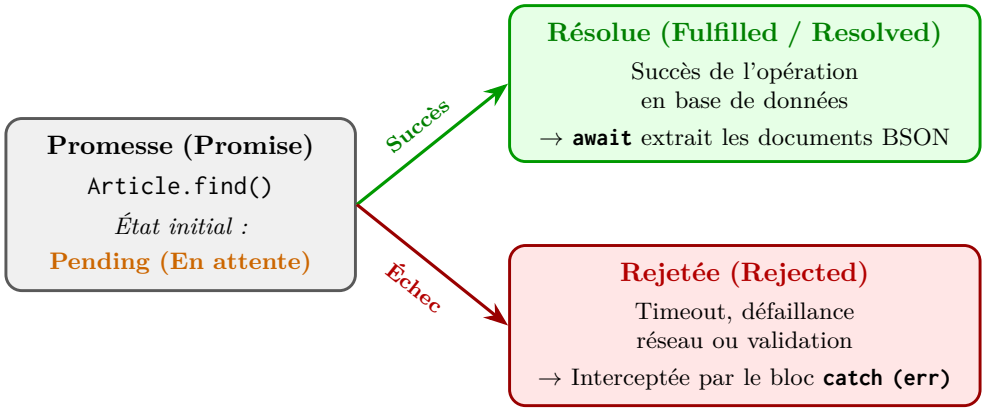


FIGURE 4 – Cycle de vie d'une Promesse et extraction synchrone via `async/await`.

Notion : `async/await` et les deux règles d'or. Le sucre syntaxique `async/await` permet d'écrire du code asynchrone avec la clarté et la fluidité d'un code synchrone :

1. Le mot-clé `await` ne peut être utilisé qu'**à l'intérieur** d'une fonction déclarée avec `async`.
2. `await` suspend temporairement l'exécution de la fonction locale en attendant que la Promesse soit résolue, **sans jamais bloquer** le fil d'exécution de Node.js pour les autres clients.
3. Tout appel avec `await` doit être encapsulé dans un bloc `try { ... } catch (err) { ... }` pour intercepter les erreurs sans provoquer l'arrêt brutal du serveur.

5 TP Guidé : Connexion et Refactorisation Pas à Pas

Étape 1 : Installation des dépendances et sécurisation

Dans le terminal de votre projet, installez l'ODM Mongoose et la bibliothèque de gestion d'environnement `dotenv` :

```
npm install mongoose dotenv
```

Créez le fichier de configuration locale `.env` à la racine de votre projet (veillez à remplacer les identifiants par votre utilisateur et mot de passe réels créés sur Atlas) :

➥ Fichier `.env` – lancer avec –

```
PORT=3000
MONGODB_URI=mongodb+srv://admin_mern:PolyMern2026!@cluster0.xxxxx.mongodb.net/blogDB?
    retryWrites=true&w=majority
```

Ajoutez immédiatement `.env` dans votre fichier `.gitignore` :

```
echo ".env" >> .gitignore
```

Étape 2 : Centraliser la connexion dans `src/config/db.js`

Créons un module dédié pour gérer l'ouverture de la connexion asynchrone à MongoDB :

➥ Fichier `src/config/db.js` – lancer avec –

```
const mongoose = require('mongoose');

async function connectDB(uri) {
  if (!uri) {
    throw new Error("L'URI MongoDB est manquant dans les variables d'environnement (.env)"
    );
  }
  await mongoose.connect(uri, {
    autoIndex: true // Génère automatiquement les index (ex: unique) en développement
  });
  console.log('Connexion a MongoDB Atlas reussie avec succes !');
}

module.exports = connectDB;
```

Étape 3 : Configurer et démarrer le serveur dans `src/server.js`

Comme en Séance précédente, notre fichier `src/server.js` conserve la structure directe que vous connaissez : il instancie Express, monte le middleware JSON et les routeurs, puis démarre l'écoute après l'établissement de la connexion à MongoDB :

➥ Fichier `src/server.js` – lancer avec `npm run dev`

```
const express = require('express');
const dotenv = require('dotenv');
const connectDB = require('./config/db');
const articleRoutes = require('./routes/articleRoutes');
const userRoutes = require('./routes/userRoutes');

// Chargement des variables d'environnement (.env)
dotenv.config();

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware pour parser le corps des requetes en JSON
app.use(express.json());

// Montage des routeurs modulaires
app.use('/api/articles', articleRoutes);
app.use('/api/users', userRoutes);

// Connexion asynchrone à MongoDB Atlas puis démarrage de l'écoute
(async () => {
  try {
    // 1. Établissement de la connexion distante
    await connectDB(process.env.MONGODB_URI);

    // 2. Démarrage de l'écoute du serveur HTTP
    app.listen(PORT, () => {
      console.log(`Serveur MERN en ecoute sur http://localhost:${PORT}`);
    });
  } catch (err) {
    console.error('Erreur critique de demarrage du serveur :', err.message);
    process.exit(1);
  }
})();
```

Point de contrôle. Lancez `npm run dev`. Le terminal doit afficher :
`Connexion a MongoDB Atlas reussie avec succes !`
`Serveur MERN en ecoute sur http://localhost:3000`
Si une erreur d'authentification s'affiche, vérifiez votre mot de passe et l'autorisation réseau

`0.0.0.0/0` sur Atlas.

Étape 4 : Définition des Modèles Mongoose

Créons les deux modèles dans le dossier `src/models/`.

➥ Fichier `src/models/userModel.js` – lancer avec –

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Le nom est obligatoire'],
      trim: true
    },
    email: {
      type: String,
      required: [true, "L'email est obligatoire"],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, "Format d'email invalide (ex:
    aya@polytechnique.tn)"]
    },
    age: {
      type: Number,
      min: [4, "L'age minimum autorise est de 4 ans"]
    },
    role: {
      type: String,
      enum: {
        values: ['user', 'admin'],
        message: '{VALUE} n\'est pas un role autorise'
      },
      default: 'user'
    }
  },
  { timestamps: true } // Ajoute automatiquement createdAt et updatedAt
);

module.exports = mongoose.model('User', userSchema);
```

➥ Fichier `src/models/articleModel.js` – lancer avec –

```
const mongoose = require('mongoose');

const articleSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Le titre est obligatoire'],
      trim: true
    },
    content: {
      type: String,
      required: [true, 'Le contenu est obligatoire']
    },
    author: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: [true, "L'auteur (authorId) est obligatoire"]
    }
  },
  { timestamps: true }
```

```
);

module.exports = mongoose.model('Article', articleSchema);
```

Notion : L'option `timestamps: true`. En passant `{ timestamps: true }` en deuxième argument du schéma, Mongoose injecte automatiquement deux dates : `createdAt` (date de création du document) et `updatedAt` (date de dernière modification). C'est un standard incontournable en entreprise pour assurer la traçabilité des données.

Étape 5 : Migration Asynchrone des Contrôleurs

Remplaçons les tableaux en mémoire par les requêtes Mongoose asynchrones.

➥ Fichier `src/controllers/userController.js` – lancer avec –

```
const User = require('../models/userModel');

// 1. Créer un nouvel utilisateur (POST /api/users)
const createUser = async (req, res) => {
  try {
    const { name, email, age, role } = req.body;
    if (!name || !email) {
      return res.status(400).json({ message: 'Le nom et l\'email sont requis' });
    }
    // Vérification de doublon d'email
    const exists = await User.findOne({ email });
    if (exists) {
      return res.status(409).json({ message: 'Cet email est deja utilise' });
    }
    const user = await User.create({ name, email, age, role });
    res.status(201).json(user);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 2. Obtenir tous les utilisateurs (GET /api/users)
const getUsers = async (req, res) => {
  try {
    const users = await User.find();
    res.status(200).json(users);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 3. Obtenir un utilisateur par son identifiant (GET /api/users/:id)
const getUserById = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ message: 'Utilisateur non trouve' });
    }
    res.status(200).json(user);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

module.exports = { createUser, getUsers, getUserById };
```

➥ Fichier `src/controllers/articleController.js` – lancer avec –

```
const Article = require('../models/articleModel');
```

```
const User = require('../models/userModel');

// 1. Récupérer tous les articles avec les données complètes de l'auteur
const getAllArticles = async (req, res) => {
  try {
    const articles = await Article.find().populate('author', 'name email');
    res.status(200).json(articles);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 2. Créer un nouvel article lié à un auteur
const createArticle = async (req, res) => {
  try {
    const { title, content, authorId } = req.body;
    if (!title || !content || !authorId) {
      return res.status(400).json({ message: 'title, content et authorId sont obligatoires
    ' });
    }
    // Vérifier l'existence réelle de l'auteur dans la collection 'users'
    const author = await User.findById(authorId);
    if (!author) {
      return res.status(404).json({ message: 'Auteur non trouve (identifiant authorId
    invalide)' });
    }

    const newArticle = await Article.create({ title, content, author: authorId });
    const populatedArticle = await newArticle.populate('author', 'name email');
    res.status(201).json(populatedArticle);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 3. Récupérer un article unique par son ID
const getArticleById = async (req, res) => {
  try {
    const article = await Article.findById(req.params.id).populate('author', 'name email')
    ;
    if (!article) {
      return res.status(404).json({ message: 'Article non trouve' });
    }
    res.status(200).json(article);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 4. Mettre à jour un article (PUT /api/articles/:id)
const updateArticle = async (req, res) => {
  try {
    const article = await Article.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true } // Renvoie le doc mis à jour et valide le schéma
    );
    if (!article) {
      return res.status(404).json({ message: 'Article introuvable pour la mise a jour' });
    }
    const populated = await article.populate('author', 'name email');
    res.status(200).json(populated);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};
```

```
// 5. Supprimer un article (DELETE /api/articles/:id)
const deleteArticle = async (req, res) => {
  try {
    const article = await Article.findByIdAndDelete(req.params.id);
    if (!article) {
      return res.status(404).json({ message: 'Article introuvable pour la suppression' });
    }
    res.status(200).json({ message: 'Article supprime avec succes', id: req.params.id });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

module.exports = {
  getAllArticles,
  createArticle,
  getArticleById,
  updateArticle,
  deleteArticle
};
```

Étape 6 : Validation du découplage SoC

Observez vos fichiers `routes/articleRoutes.js` et `routes/userRoutes.js` : ils n'ont subi **strictement aucune modification** ! Les routes associent toujours les chemins aux fonctions du contrôleur. Ce découplage garanti par la Séance 2 permet de remplacer un moteur de stockage complet sans casser l'architecture du projet.

Étape 7 : Validation Complète de Bout en Bout avec Postman

Exécutez ce scénario méthodique de test dans Postman :

1. **Création d'un utilisateur valide (POST `http://localhost:3000/api/users`) :**

```
{
  "name": "Aya Ben Salah",
  "email": "aya@polytechnique.tn",
  "age": 22,
  "role": "admin"
}
```

→ Statut **201 Created**. **Copiez la valeur du champ `_id`** généré par MongoDB (ex : `"665b9..."`).

2. **Test de validation des contraintes (rejet par Regex et âge minimum) :** Tentez d'insérer un document non conforme avec un format d'email erroné et un âge inférieur à 4 ans :

```
{
  "name": "Test Invalide",
  "email": "adresse_sans_arobase",
  "age": 2
}
```

→ Statut **400 Bad Request** ! Mongoose déclenche une erreur `ValidationError` et bloque l'écriture sans polluer la base de données.

3. **Création d'un article lié (POST `http://localhost:3000/api/articles`) :** Collez l'`_id` de l'utilisateur dans le champ `authorId` :

```
{
  "title": "Maîtriser MongoDB Atlas & Mongoose",
  "content": "L'ODM Mongoose simplifie grandement la validation et les relations
    NoSQL.",
  "authorId": "COLLEZ_ICI_L_ID_UTILISATEUR"
}
```

→ Statut **201 Created**. Observez la réponse JSON : le champ `author` contient directement l'objet avec `name` et `email` grâce à `.populate()` !

4. **Vérification de la persistance réelle :** Arrêtez votre serveur dans le terminal avec `Ctrl+C`, puis relancez-le avec `npm run dev`. Exécutez un `GET http://localhost:3000/api/articles` : vos données sont toujours intactes, directement lues depuis MongoDB Atlas !

Point de contrôle. Ouvrez l'interface web de MongoDB Atlas, allez dans **Database** → onglet **Collections**. Vous devez voir apparaître la base `blogDB` contenant deux collections distinctes : `users` et `articles`.

6 Exercices d'Application en Autonomie

Exercice 1 : Intégrité Référentielle et Contraintes Avancées (10 points)

Pour consolider la robustesse de l'annuaire des utilisateurs et prévenir la corruption relationnelle :

1. Enrichissez `src/models/userModel.js` avec deux nouveaux champs :
   - `bio` : Type `String`, optionnel, avec contrainte de longueur maximale `maxlength: [200, 'La biographie ne peut pas dépasser 200 caractères']` ;
   - `phone` : Type `String`, validé par une expression régulière vérifiant un numéro de téléphone valide (ex : `match: [/^\d{8}$/, 'Numéro de 8 chiffres requis']`).
2. Implémentez la suppression sécurisée d'un utilisateur (`DELETE /api/users/:id`) dans le contrôleur `controllers/userController.js` et déclarez la route associée dans

`routes/userRoutes.js`.

3. **Vérification d'intégrité relationnelle :** Avant de supprimer un utilisateur, vérifiez l'existence de dépendances avec `Article.findOne({ author: req.params.id })`. Si l'utilisateur possède au moins un article publié, interdisez la suppression en renvoyant un statut `400 Bad Request` avec le message : *"Impossible de supprimer un utilisateur ayant des articles publies"*.

Exercice 2 : Gestion Professionnelle des Identifiants Invalides (CastError) (6 points)

Dans MongoDB, les identifiants `_id` sont des chaînes hexadécimales de 24 caractères (ex. `665a1b2c3d4e5f6a7b8c9d0e`). Si un client Postman envoie un identifiant mal formé (ex : `GET /api/articles/123`), Mongoose lève une exception de type `CastError` interceptée dans le bloc `catch (err)`.

1. Dans vos fonctions de contrôleur recevant un identifiant (`getArticleById`, `getUserById`), examinez le bloc `catch (err)`.
2. Détectez si l'erreur interceptée possède le nom `err.name === 'CastError'`.
3. Si tel est le cas, répondez avec le statut **400 Bad Request** et le message JSON suivant :

```
{ "message": "Identifiant de ressource invalide (format ObjectId attendu)" }
```

   sinon, répondez avec le statut **500** et le message d'erreur.
4. Testez avec Postman en envoyant un GET sur `/api/articles/abc` et vérifiez que vous recevez bien une erreur 400 propre au lieu d'une 500.

Contenu du compte-rendu

Chaque étudiant remet **un seul fichier PDF individuel** sur la plateforme pédagogique contenant :

1. Votre réponse rédigée et argumentée à l'encadré **Prédisez**.
2. Les captures d'écran de l'interface MongoDB Atlas montrant vos deux collections (`users` et `articles`) avec des documents insérés.
3. Les captures d'écran Postman démontrant :
   - La création d'un utilisateur et la création d'un article lié ;
   - La consultation `GET /api/articles` avec l'auteur peuplé via `.populate()` ;
   - Le test de tentative de doublon d'email (renvoyant le statut `409 Conflict`) ;
   - Le test de l'exercice 2 avec un identifiant mal formé (renvoyant le statut `400 Bad Request`).
4. Le code source complet de vos modèles et de vos contrôleurs enrichis.

Date limite de dépôt : La veille de la Séance 4 à 23h59 sur la plateforme de l'école. Tout retard entraînera une pénalité.

7 Récapitulatif

Six idées fondamentales à retenir

1. La mémoire vive (RAM) est éphémère ; une base de données cloud comme **MongoDB Atlas** assure la persistance, l'intégrité et la scalabilité des données.
2. Toute opération de base de données est une entrée/sortie réseau asynchrone ; elle renvoie une **Promesse** que l'on manipule proprement avec le duo **`async / await`**.
3. Les fonctions asynchrones doivent être protégées par des blocs **`try ... catch`** afin d'intercepter les erreurs et d'éviter l'arrêt du processus serveur.
4. **Mongoose** structure l'univers NoSQL grâce aux **Schémas** (règles et types de validation) qui sont compilés en **Modèles** (constructeurs dotés de méthodes CRUD).
5. Les relations NoSQL s'expriment par des références d'identifiants (`type: ObjectId, ref: 'User'`) et sont résolues dynamiquement grâce à **`.populate()`**.
6. Les secrets d'accès à la base de données ne doivent jamais figurer dans le code source : on utilise le module **`dotenv`** et le fichier **`.env`** exclu par **`.gitignore`**.

Je veux...	**J'écris...**
Connecter mon serveur à MongoDB Atlas	`await mongoose.connect(process.env.MONGODB_URI);`
Définir un champ de texte obligatoire et nettoyé	`title: { type: String, required: true, trim: true }`
Définir une référence relationnelle vers User	`author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }`
Récupérer tous les documents avec jointure virtuelle	`await Article.find().populate('author', 'name email');`
Trouver un document par son identifiant	`const article = await Article.findById(req.params.id);`
Créer et insérer un document validé	`const newDoc = await Article.create(req.body);`
Mettre à jour et récupérer le nouveau document	`await Article.findByIdAndUpdate(id, data, { new: true });`
Supprimer définitivement un document	`await Article.findByIdAndDelete(req.params.id);`

Je sais faire (auto-évaluation)

☐	Expliquer la différence entre mémoire vive volatile et stockage persistant	Section 1
☐	Déployer un cluster MongoDB Atlas, configurer les utilisateurs et l'accès réseau	Section 2
☐	Sécuriser les secrets de connexion avec `dotenv` et `.gitignore`	Section 2
☐	Concevoir des Schémas et compiler des Modèles avec Mongoose	Section 3
☐	Remplacer le CRUD en mémoire par des appels asynchrones avec `async/await`	Section 5
☐	Reconstituer une relation NoSQL entre deux collections via `.populate()`	Section 5
☐	Gérer les erreurs de validation, de doublon d'index (11000) et d'ID invalide (`CastError`)	Section 6

8 Références des Notions de la Séance (Session Concept Reference)

1. Méthodes Indispensables du Modèle Mongoose

Méthode Mongoose	Rôle dans l'API REST	Exemple d'écriture
`Model.find(filtre)`	Récupère une liste de documents (tous si filtre vide)	`await Article.find({ author: id });`
`Model.findById(id)`	Récupère un document unique par sa clé primaire `_id`	`await Article.findById(id);`
`Model.findOne(filtre)`	Récupère le premier document validant le critère	`await User.findOne({ email });`
`Model.create(donnees)`	Valide et insère un ou plusieurs nouveaux documents	`await User.create(req.body);`
`Model.findByIdAndUpdate`	Modifie un document existant (`opt: { new: true }`)	`await Article.findByIdAndUpdate(id, data, { new: true });`
`Model.findByIdAndDelete`	Supprime le document désigné par son identifiant	`await Article.findByIdAndDelete(id);`
`query.populate('champ')`	Remplace un `ObjectId` par le document lié	`await Article.find().populate('author', 'name email');`
`Model.countDocuments()`	Dénombre efficacement les documents d'une collection	`await Article.countDocuments();`

2. Programmation Asynchrone en Node.js

Concept / Syntaxe	Signification technique	Exemple d'usage
`async function`	Déclare une fonction asynchrone qui renvoie une Promesse	`const getAll = async (req, res) => { ... };`
`await expression`	Met en pause l'exécution locale jusqu'à résolution	`const data = await Model.find();`
`try { ... } catch (err)`	Capture toute erreur asynchrone (évite le crash du serveur)	`try { ... } catch (err) { res.status(500).json(...) }`
`Promise.all([p1, p2])`	Exécute plusieurs Promesses en parallèle et attend la fin	`const [arts, users] = await Promise.all([...]);`
`new Promise((res, rej))`	Crée une Promesse personnalisée avec gestion succès/échec	`return new Promise((resolve, reject) => ...);`

3. Types et Validateurs de Schéma Mongoose

Option de Schéma	Rôle de validation	Exemple concret
`type: String / Number`	Typage strict des propriétés du document	`title: { type: String }`
`type: ObjectId, ref`	Définition d'une clé étrangère vers une autre collection	`author: { type: ObjectId, ref: 'User' }`
`required: [true, '...']`	Rend la présence du champ obligatoire à l'insertion	`email: { required: [true, 'Email requis'] }`
`unique: true`	Crée un index d'unicité dans MongoDB (rejette doublons)	`email: { type: String, unique: true }`
`match: [/regex/, '...']`	Valide le format textuel par une expression régulière	`email: { match: [/^\S+@\S+\.\S+$/] }`
`min: [4, '...'] / max`	Impose des bornes numériques minimales et maximales	`age: { type: Number, min: 4 }`
`trim: true`	Supprime automatiquement les espaces en début et fin	`name: { type: String, trim: true }`
`lowercase: true`	Convertit la chaîne en minuscules avant enregistrement	`email: { type: String, lowercase: true }`
`enum: ['user', 'admin']`	Restreint la valeur à un ensemble fini de chaînes	`role: { type: String, enum: ['user', 'admin'] }`
`default: valeur`	Attribue une valeur par défaut si le champ est omis	`role: { type: String, default: 'user' }`
`{ timestamps: true }`	Crée automatiquement les champs `createdAt` & `updatedAt`	`new mongoose.Schema({ ... }, { timestamps: true });`

4. Expressions Régulières (Regex) pour la Validation d'API

Symbole / Motif	Signification & Fonctionnement	Exemple d'Application API
`^` et `$`	Début (`^`) et fin (`$`) stricts de chaîne	`/^aya$/` correspond uniquement à "aya"
`[a-zA-Z0-9]`	Classe de caractères autorisés	Identifiant : `/^[a-zA-Z0-9_-]{3,16}$/`
`[^...]`	Négation : exclut les caractères listés	`[^\s@]+` (aucun espace ni arobase)
`\d`	Chiffre décimal équivalent à `[0-9]`	Téléphone : `/^\d{8}$/` (8 chiffres)
`\w`	Caractère de mot (`[a-zA-Z0-9_]`)	Slug d'article : `/^[\w-]+$/`
`\s` / `\S`	Espace blanc / Non-espace blanc	Mot de passe sans espace : `/^\S+$/`
`+` / `*` / `?`	Quantificateurs : 1 ou +, 0 ou +, 0 ou 1	Indicatif optionnel : `/(\+216)?\d{8}/`
`{n,m}`	Répétition bornée (entre n et m fois)	Mot de passe : `/^\S{8,32}$/`
Motif Email	Structure standard `identifiant@domaine.ext`	`/^\S+@\S+\.\S+$/`
`match: [/reg/, '...']`	Propriété Mongoose exécutant le regex	Déclenche `ValidationError` (400) si non conforme

5. Configuration d'Environnement et Sécurité (`dotenv`)

Élément	Rôle dans l'application	Bonne pratique professionnelle
`dotenv.config()`	Charge les variables du fichier `.env` dans `process.env`	À appeler au tout début de `server.js` avant tout code
`process.env.NOM`	Accède en lecture à une variable d'environnement	`const PORT = process.env.PORT || 3000;`
Fichier `.env`	Stocke localement les secrets (mots de passe, clés, URI)	Ne jamais commiter dans Git (à placer dans `.gitignore`)
Fichier `.env.example`	Modèle documenté sans aucun mot de passe secret	Exemple : `MONGODB_URI=mongodb+srv://...`
URI `mongodb+srv://`	Chaîne de connexion et routage DNS vers le cluster Atlas	Toujours vérifier l'URL-encoding des caractères du mot de passe

La prochaine fois (Séance 4 : Sécurisation de l'API avec JWT & Authentification). Maintenant que nos utilisateurs et articles sont stockés de façon permanente et relationnelle dans MongoDB, nous allons protéger les routes de notre API. Nous aborderons le hachage sécurisé des mots de passe avec **bcrypt**, la génération de jetons d'authentification **JSON Web Tokens (JWT)** et la création de **middlewares d'authentification** pour restreindre la création et la modification d'articles aux seuls utilisateurs connectés !